Lecture 8: Network Parallelism

Network the Topology

- Understand the systems complexities of training huge models.
- Different parallelization paradigms and why people use multiple at once.
- What large scale training runs often look like?

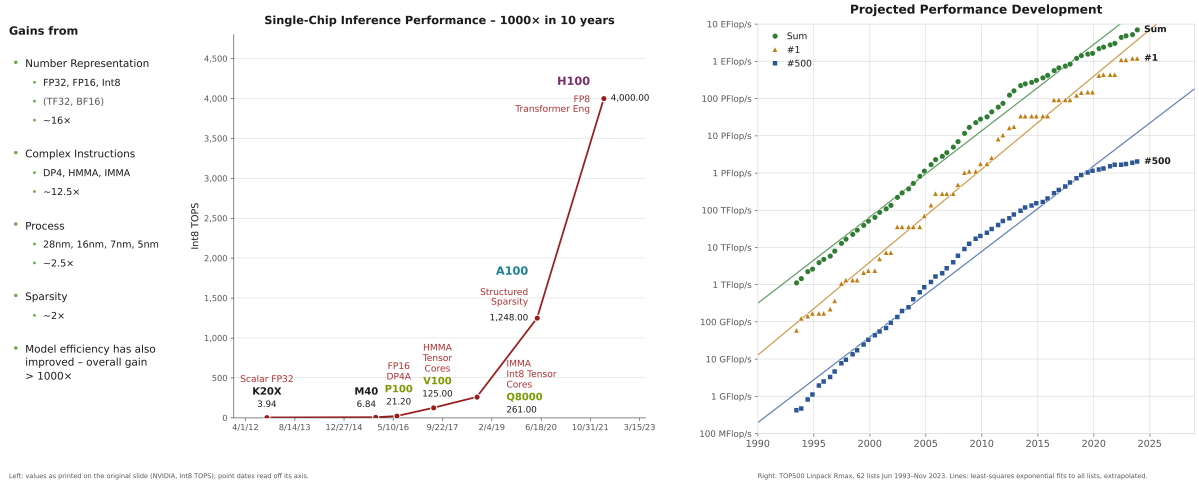
Organization today

Part 1: Basics of networking for LLMs

Part 2: Different forms of parallel LLM training

Part 3: Scaling and training big LMs with parallelism

Limits to GPU scaling - compute



There are limits to single-GPU scaling.

The world's fastest supercomputers have *exaflops* of compute.

Important detail : all-reduce vs reduce-scatter-gather

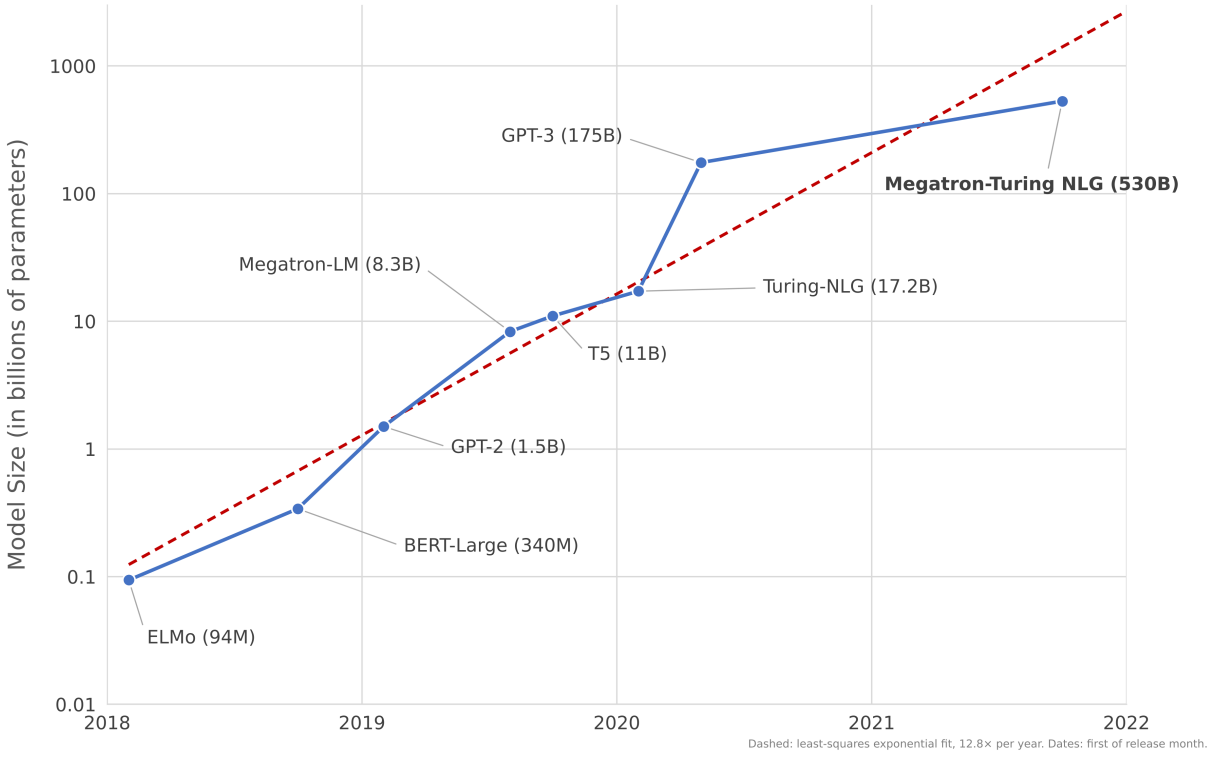
Reduces can be implemented as two steps:

$$\text{All-reduce} = \text{Reduce-scatter} + \text{All-gather}.$$

Importantly, in the bandwidth-limited regime, this is the best you can do.

Limit to GPU scaling - memory

Models are getting really big ..



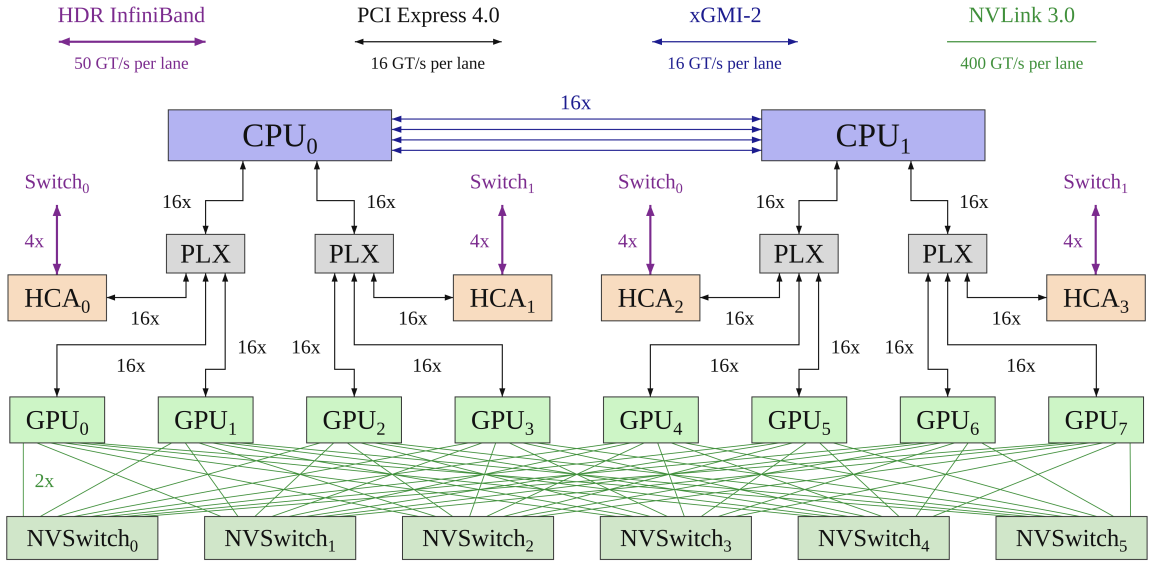
A single GPU can't fit most of these large models!

GPUs vs TPUs - design differences at the communication level

- **GPU networking** = all-to-all (up to 256).
- **TPU networking** = toroidal mesh.

What do we do? Multi-GPU, multi-machine parallelism

Intra-node parallelism (high-speed). Inter-node parallelism (slow-speed).



Split up compute and memory requirements across GPUs and machines.

Why tree? Why mesh?

Pro tree / A2A - Better for less structured communications (expert parallel).

Pro mesh - Cheaper to operate, can be made fast (and just do tensor parallel).

But then things changed .. TPU8i/t

TPU8i networking - closer to tree-style topologies (maybe for MoEs?).

TPU8t scale-out networking - ”Virgo” with switched networks.

Why not connect everything?

Key Capabilities - Huawei Ascend 910C Cloud Matrix 384 = SRAM Everywhere!

New unit of compute - the datacenter.

What we want from multi-machine scaling:

- Linear compute scaling (model FLOPs scale linearly with num GPUs).
- Linear memory scaling (max model params scales with num GPUs)

Standard LLM parallelization primitives

How do we parallelize LLMs? 3 important ideas!

- **Data parallelism**
  - Naive data parallel
  - ZeRO levels 1-3
- **Model parallelism**
  - Pipeline parallel
  - Tensor parallel
- **Activation parallelism**
  - Sequence parallelism
  - Context parallelism

Naive data parallelism

Starting point - imagine we are doing naive SGD

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^{B} \nabla f(x_i)$$

Naive parallelism: split the elements of B sized batch across M machines and exchange gradients to synchronize.

Question: *How does this do?*

- Compute scaling - each GPU gets B/M examples.
- Communication overhead - transmits $2\times$ # params every batch. OK if batches are big.
- Memory scaling - every GPU needs # params at least.

What's wrong with naive data parallel?

Memory seems like it'd be a problem - we copy the model parameters to each GPU. Let's take a closer look ..

Claim: *Our memory situation is actually terrible.*

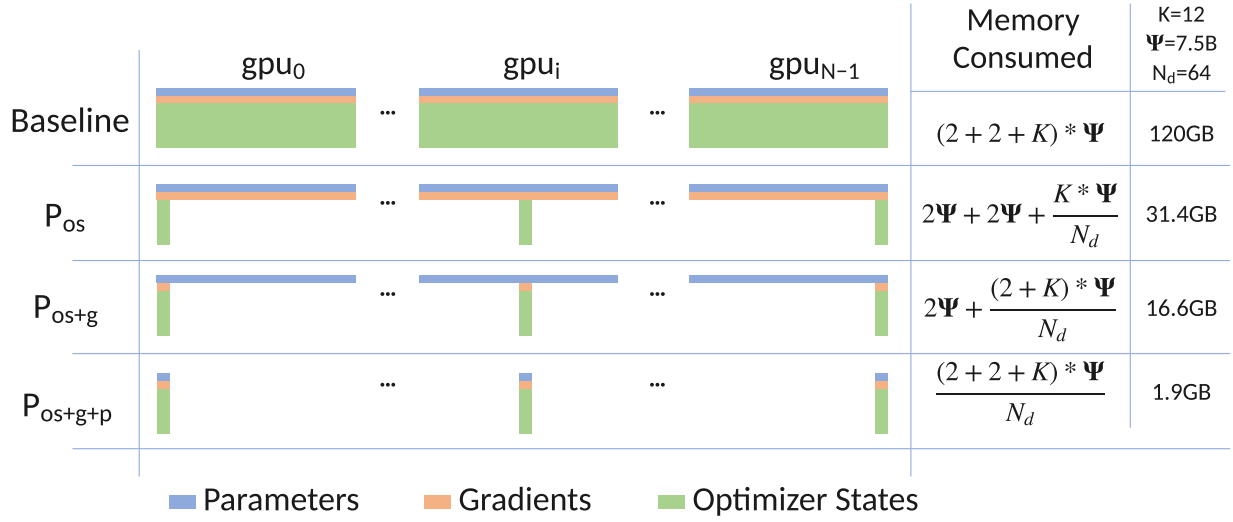
Depending on our precision ..

We need 5 copies of weights and 16 bytes per param!

- 2 bytes for FP/BF16 model parameters
- 2 bytes for FP/BF16 gradients
- 4 bytes for FP32 master weights (the thing you accumulate into in SGD)
- 4 (or 2) bytes for FP32/BF16 Adam first moment estimates
- 4 (or 2) bytes for FP32/BF16 Adam second moment estimates

ZeRO - solving the memory overhead issue of DP

Core idea: split up the expensive parts (state) and use the reduce-scatter equivalence.



ZeRO stage 1. optimizer state sharding

High level idea:

- Split up the optimizer state (first + second gradients) across GPUs.
- Everyone has the parameters + gradients.

Each worker is responsible for updating a subset of params (corresponding to their slice).

Step 1. Everyone computes a full gradient on their subset of the batch.

Step 2. ReduceScatter the gradients - incur # params communication cost.

Step 3. Each machine updates their param using their gradient + state.

Step 4. AllGather the parameters - incur # params communication cost.

Claim: *ZeRO stage 1 is free (in the bandwidth limited regime) memory wins.*

ZeRO stage 2. the simple extension to gradient sharding

Emboldened by our success, let's shard even more stuff.

High level idea:

- Also keep the gradients (pink slices) sharded across the machines.
- Use the same (rough) tricks as stage 1.

Complexity - we can never instaniate a full gradient vector, but each worker must compute a full gardient (since we're data parallel).

Step 1. After computing a layer's gradients, immediately reduce to send this to the right worker.

Step 2. Once the gradients are not needed in the backward graph, immediately free it.

Step 3. Each machine updates their param using their gradient + state.

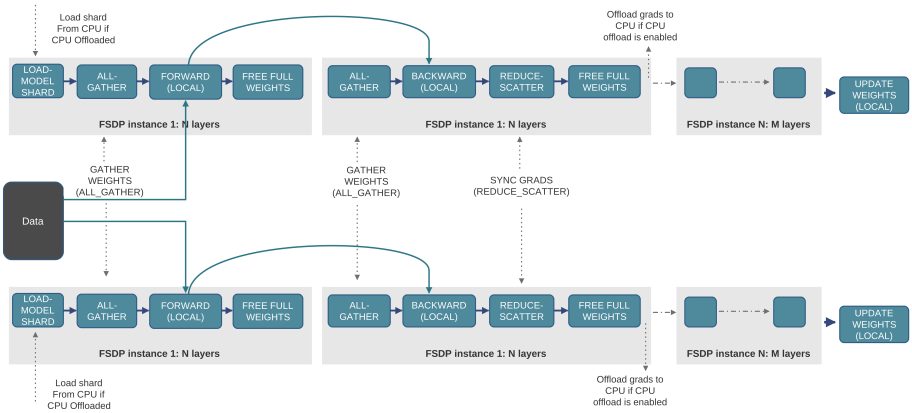
Step 4. AllGather the parameters.

ZeRO stage 3 (aka FSDP) shard *everything*

We've gotten almost everything for free so far .. lets try to solve *all* our memory issues.

High level idea:

- Shard everything - incl parameters!
- Use the same "incremental computation / communication" ideas.
- Send and request parameters on demand while stepping through the compute graph.



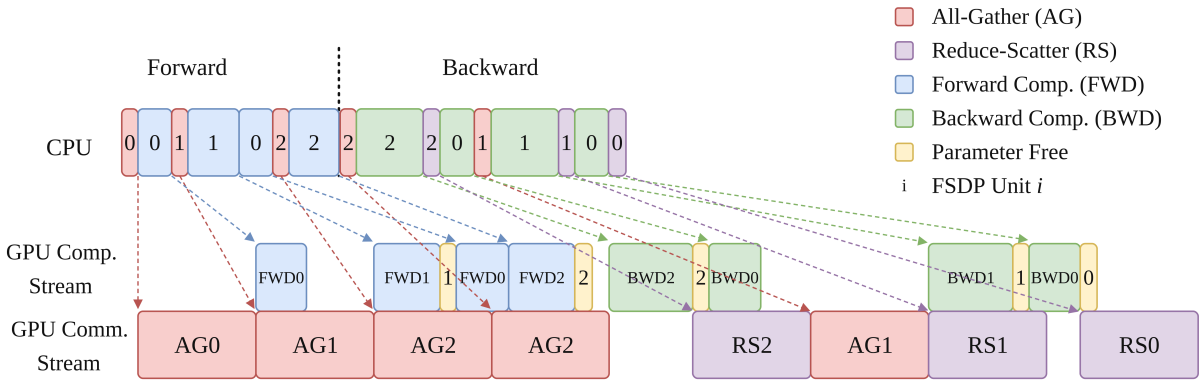
Communication cost - 2 AllGather (# param), 1 ReduceScatter (# param).

Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed.

Overlapping communication and computation

- The all-gathers happen all at once while forward happens, masking the communication cost.



Distributed data parallel costs $2 * \#$ param communication.

- **Zero stage 1** is $2 * \#$ param - it's free! - you might as well always do it.
- **Zero stage 2** is $2 * \#$ param - it's (almost) free (ignoring overhead).
- **Zero stage 3** is $3 * \#$ param - $1.5\times$ comm cost, but that's not bad (ignoring latency..)!

This is also conceptually very simple - write a FSDP block wrapper.

Issue remain with data parallel - computing scaling

With data parallel, # machines $<$ batch size (and near this, comm overhead is high).

And there's diminishing returns to batch sizes.

Issue remain with data parallel - models don't fit

Zero stages 1 and 2 don't let you scale memory.

Zero stage 3 is nice in principle, but *does not reduce activation memory*.

Better ways to split up the model is needed ...

Beyond data parallel - model parallelism

Scaling up in memory (without changing batch size) with model parallelism!

- It splits up the parameters across GPUs (like ZeRO 3) ..
- But communicate activations (while ZeRO 3 sends params) ..

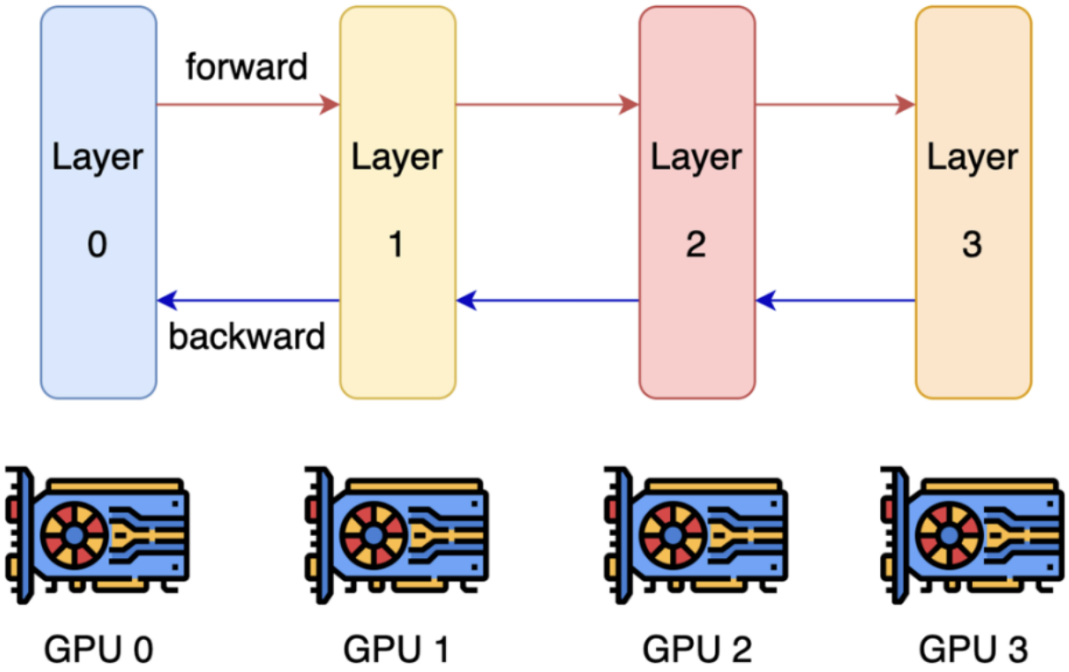
We cover 3 different types of parallelism!

1. Pipeline parallelism
2. Tensor parallelism (+ sequential parallelism)
3. Expert parallelism

These correspond to three different ways of cutting up the model.

What's layer-wise parallel?

Layer-wide parallel cuts up layers, assigns some subset to GPUs.

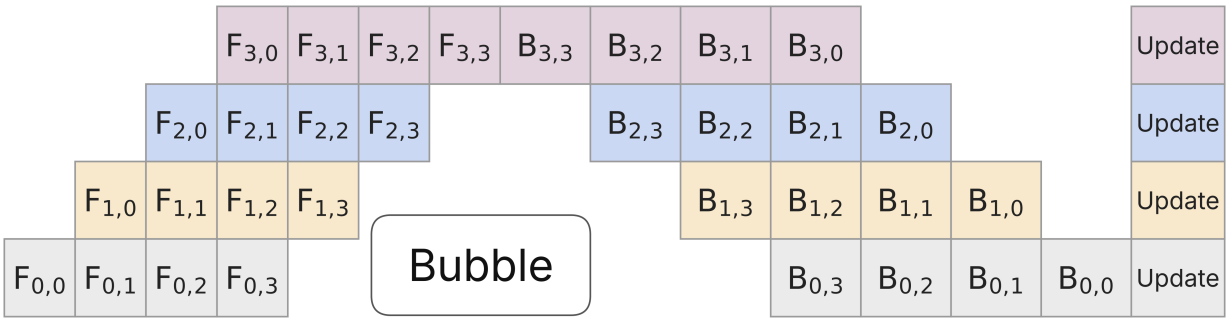


Activations and partial gradients are passed back and forth.

What's wrong with layer-wise parallel?

Utilization of layer-wise parallelism is *terrible*..

With n GPUs, each GPU is active $\frac{1}{n}$ of the time.



Solution: Pipeline-parallel.

Process "micro-batches" (in this case, 4).

Send off the first microbatch and start computing the second.

The ratio of bubble time to useful compute compute is

$$\frac{n_{\text{stage}} - 1}{n_{\text{micro}}}$$

so we need a big batch size!

Why pipeline parallel?

Pipelines seem terrible. Why do we do it?

1. Pipelines save memory (compared to DDP)
2. Pipelines can have good communication properties (compared to FDSP) - it depends only on activations ($b \times s \times h$) and is *point to point*

Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

- Pipeline performance is highly dependent on batch size
- Batch sizes are key to hiding the buddle - otherwise pipeline rapidly degrades performance
- Trade communication bandwidth for utilization
- Some more crazy pipeline patterns can improve utilization, but at the cost of bandwidth

"Zero bubble" pipelining

Split up backwards into 2 parts

1. Backpropagating activations (x, z)
2. Computing weight gradients (W)

Claim: *The second part can be done whenever.*

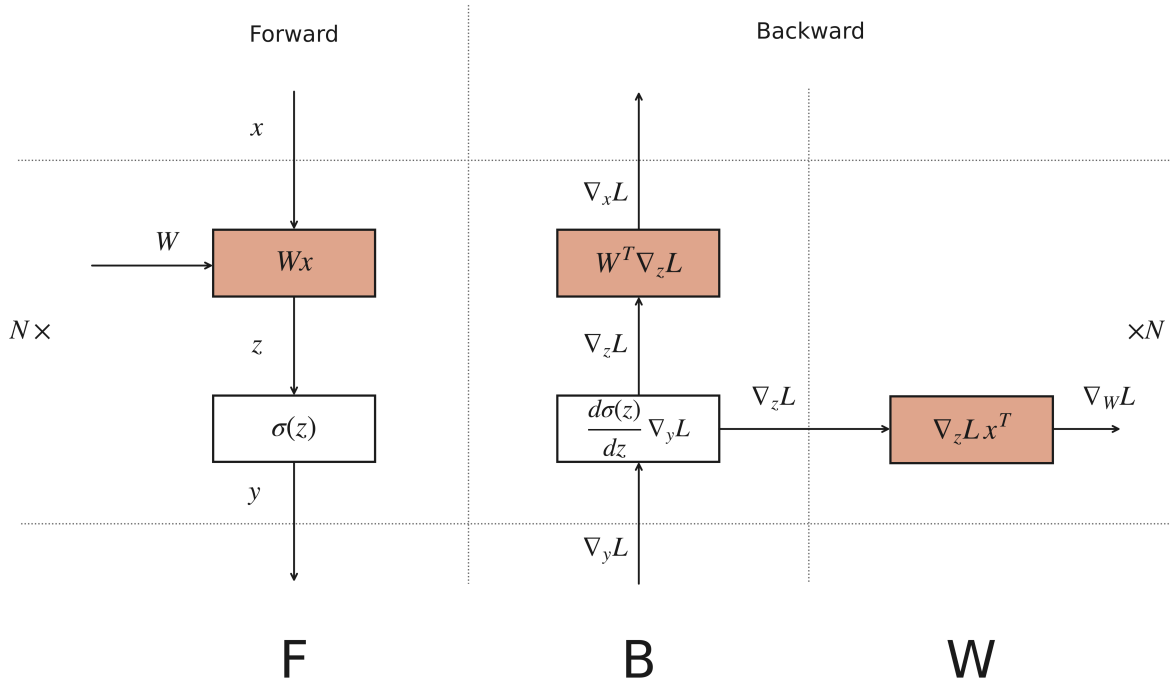


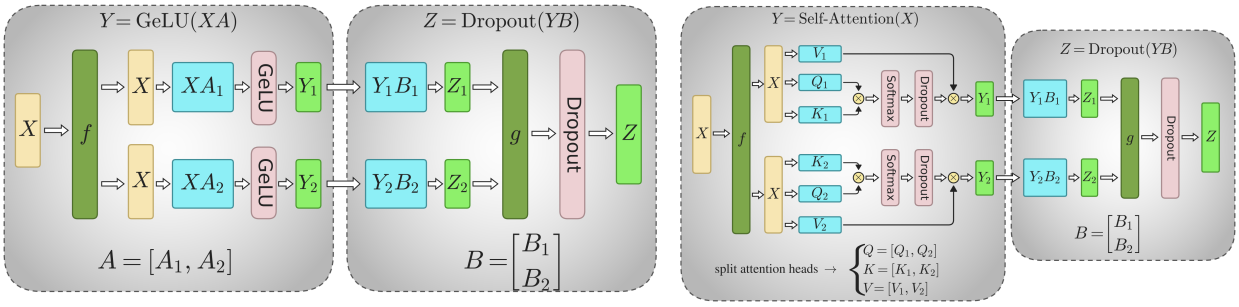
Figure 1: Computation Graph for MLP.

Model parallel along the width axes

Question: *Are there model parallel schemes with better utilization?*

- We can think of pipeline parallel as cutting up along depth. What about width?
- Simple matrix multiplication observation: decompose into submatrices, add partial sums.

Tensor parallel - GPUs have submatrices



(a) MLP (b) Self-Attention

Assign columns ($A1, A2$) and rows ($B1, B2$) to separate GPUs.

- In the forward pass, f is the identity and g is an all-reduce.
- In the backward pass, f is an all-reduce, g is the identity.

How do we split up in tensor parallel across a transformer block?

- Columnwise - QKV, up-projection
- Rowwise - Attention output, down-projection
- Replicated - norms, routers, etc

When do we tensor parallel?

On GPUs, tensor parallel within a node (up to 8 GPUs) due to high speed interconnects.

How do things compare to pipeline parallel?

Pros

- No bubble. If your network is fast enough, there's no waiting for others.
- Low-complexity. It is simple to "wrap" models without major infrastructure changes.
- Doesn't need large batch size to work well.

Cons

- Much larger communication than pipeline parallel.
- Pipeline: bsh point-to-point communication per microbatch.
- Tensor: $8bsh\left(\frac{n_{\text{devices}}-1}{n_{\text{devices}}}\right)$ per layer and all-reduce communication.

Use tensor parallel whenever we have low-latency, high-bandwidth interconnects.

A final complexity - memory is dynamic!

Memory isn't just the static bits, but also activations! This can be very large!

- Thus far, we have only really discussed parameter memory.
- Tensor and pipeline parallel can linearly reduce those .. **but what about activations?**

What's the activation memory per layer?

Starting point: activation memory needed if storing everything.

$$\text{Activations memory per layer} = sbh\left(34 + 5\frac{as}{h}\right)$$

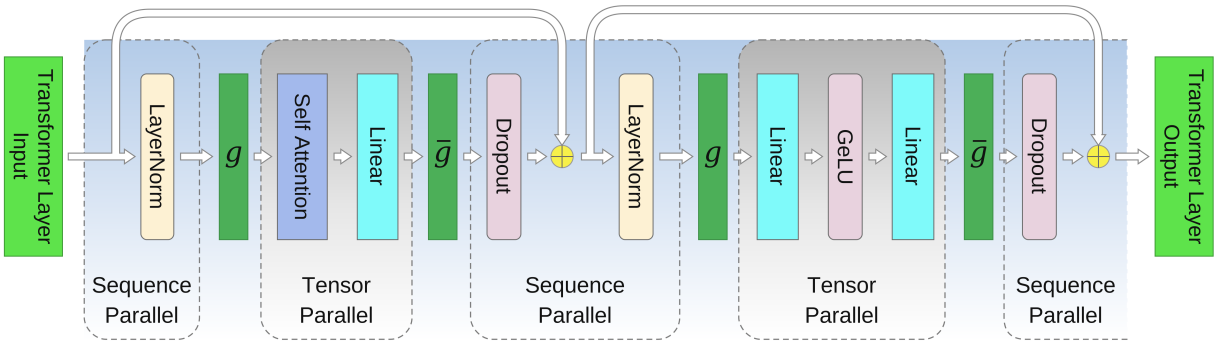
- The $5\frac{as}{h}$ term comes from the quadratic attention terms including dropout.
- As with flash attention, we can drop this term via recomputation.

Next point: activation memory under tensor parallel.

$$\text{Activations memory per layer} = sbh\left(10 + \frac{24}{t} + 5\frac{as}{ht}\right)$$

- Tensor parallel splits the matrix multiples in attention + MLP.
- The remaining 10 term is for the LayerNorm ($4sbh$), Dropout ($2sbh$), and inputs to attention and MLP ($4sbh$).
- These terms alone will continue to grow with size.

Making memory truly linear - sequence parallel



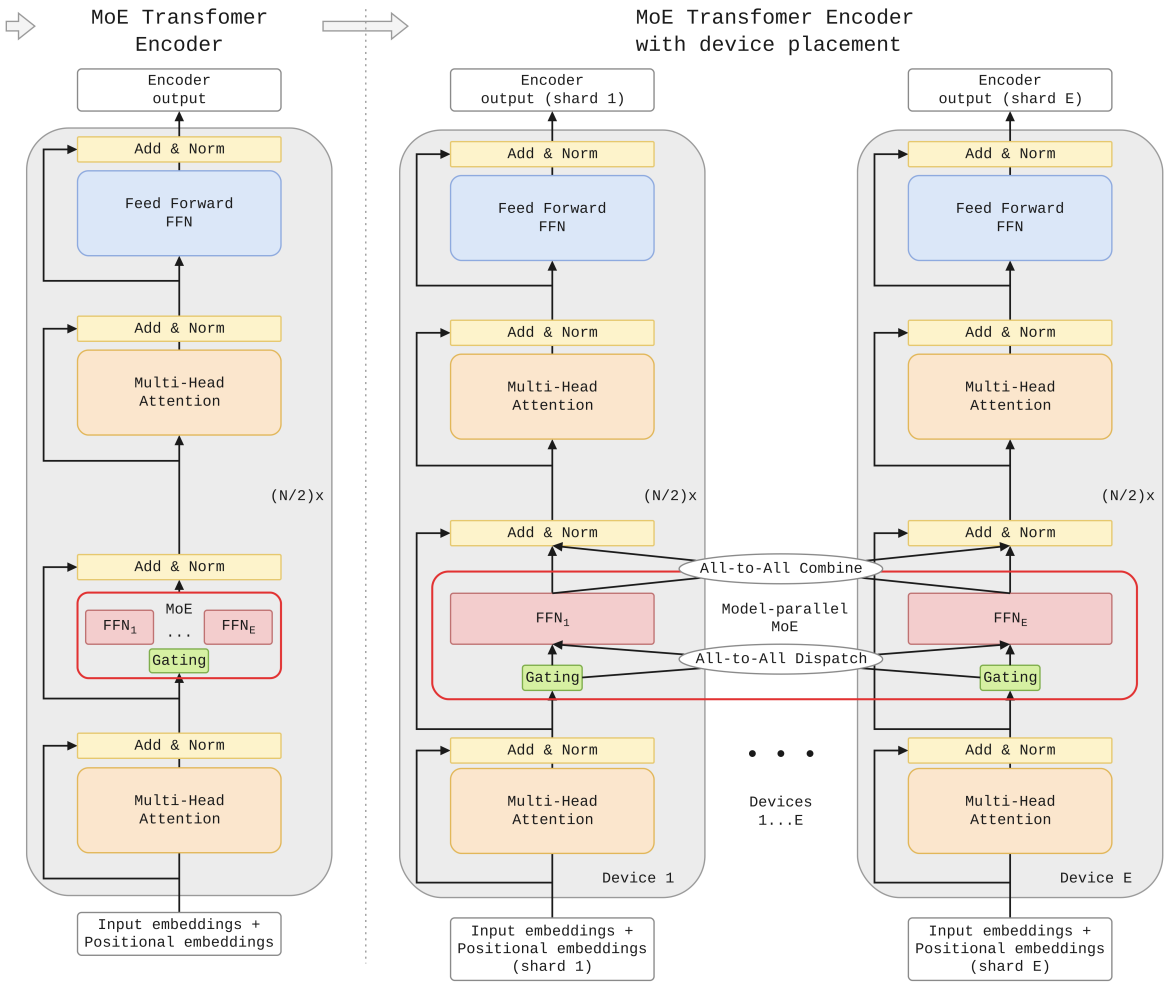
Observation: all the $10sbh$ terms are pointwise operations over the sequence .. so split up the layer norm/dropout layers along the sequence axis.

- In the forward pass, 'g' is an all-gather and '$\overline{g}$' is reduce-scatter.
- In the backward pass, 'g' and '$\overline{g}$' are reversed.

Making activation memory fully scale with more machines.

Expert parallelism

Instead of splitting up the matmul, split up the expertd and route activations.



Putting it together to get full linear scaling for memory.

Why EP?

EP is *roughly* like TP in behavior for MLPs - high bandwidth, reduces activation.

Guideline 4: Prefer EP over TP for Expert Layers

EP Advantages	Details
Better GEMM efficiency	Larger local matrix sizes improve GPU utilization
Lower communication	EP has less communication overhead than TP for MoE layers
Simpler computation graph	Easier to overlap communication with computation
Token permutation	When `EP = num_experts`, local token permutation is eliminated

Example: For Mixtral 8x7B, `EP8×TP1` outperforms `EP4×TP2`.

But, splitting matmuls can reduce efficiency verse routing activations.

Parallelism Complexity

Naively - we can compose EP and all other strategies .. **but there are important notes** .. DP usually shares replicas with EP splits (so EP $<$ DP).

Decoupling attention / expert parallelism

This can lead to some imbalance between attention and MLPs for parallelism:

- High TP - useful for attention parallelization (since we can't use EP).
- Low TP - useful for MLP parallelization (we'd rather EP than TP).

Context parallel / ring attention (split activations across GPUs in a long sequence)

Method	Comm/Sync	Param memory per rank	Activation / KV memory per rank	Main bandwidth cost	Scales global batch?	Easy to use?
DDP / ZeRO-1	gradient all-reduce per step	**No param scaling** (ZeRO-1 only opt state)	**None**	**Gradient traffic ~ O(params)**	**Yes, linear in DP**	**Very**
FSDP / ZeRO-3	Gradient all reduce – can be overlapped	**~1/DP** for params/grads/optimizer state	**None**	**Param traffic ~ O(params)**, higher than DDP, overlap	**Yes, linear in DP**	**Moderate**
Pipeline parallel	Activation between layers, **pipeline bubbles**	**~1/PP**	Depends on pipeline buffers	**Activation traffic between stages**	**No**, but needs microbatches	**Hard**
Tensor parallel	**Blocking activation communication**	**~1/TP** for TP-sharded weights	**~1/TP** for relevant matmul activations w/ SP	**Activation-sized collectives every block**	**No**	**Hard**
Sequence / Context parallel	per-layer sequence-shard exchange	**None**	**~1/SP or 1/CP** on sequence-side activations / KV	**Activation / KV communication**	**No**	**Hard**
Expert parallel (MoE)	**token dispatch - all-to-all per MoE**	**~1/EP for expert weights only**	**None**	**Token-routing all-to-all**	**No**, but needs enough tokens per expert	**Hard**

3D (4D) complexity - putting it all together

Simple rules of thumb from the literature.

1. Until your model fits in memory ..
    - tensor / expert parallel up to GPUs / machine
    - pipeline parallel across machines
    - (or use Zero-3, depending on BW)
2. Then until you run out of GPUs ..
    - scale the rest of the way with data parallel
    - if your batch size is small .. gradient accumulate to trade higher batch sizes for better communication efficiency

